

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-afbd95b058dfcc672.jsonl`  
- SHA-256: `4f90a741de86686980bd22140b6e272f8949aa432208c23fd60bd0cd690d458b`  
- Source modified: `2026-06-13T19:17:19+00:00`  
- Imported at: `2026-07-08T16:00:00+00:00`  
- project: `wf_095035a0-e9f`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-13T19:15:44.266Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

FINDING: {"title":"person_in_court degenerate-dims assertion pins an implementation artifact, not meaningful behavior", "severity":"low", "file":"tests/test_fusion_features.py:46-47", "detail":"Line 47 asserts person_in_court(in_box, 0, 0, LEFT_HALF) is True with the comment '(0,0) is on left half edge-inside'. This is incidental: with frame dims 0, _foot_norm returns (0.0,0.0), and (0,0) happens to be classified inside LEFT_HALF by the ray-cast. The test reads like it's verifying graceful degradation, but it's actually asserting that a person with unusable frame dims is silently counted as on-court at the origin ? a behavior that, combined with finding #2 (fh=0 collapse), is arguably wrong (a detection with no valid coordinates probably should not count as in-court). The assertion is not vacuous but it locks in a dubious artifact as if it were intended.", "fix":"Decide the intended degenerate-dims contract (count nobody vs count everybody) and assert that explicitly with a comment explaining why, rather than asserting whatever the origin happens to ray-cast to. If origin-inside is genuinely fine, say so; if not, fix _foot_norm/person_in_court to treat bad dims as not-in-court."}

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:15:47.420Z)

I'll verify this finding by reading the actual test code and the underlying implementation.

3. user (2026-06-13T19:15:48.375Z)

```
1      """Pure-logic unit tests for the person-cue fusion features  
(backend.pipeline.detectors.fusion_features).  
2  
3      No GPU/torch/cv2 ? synthetic person boxes + shuttle points exercise every feature path  
and  
4      edge case. This is the testable core that proves the fusion math correct before any GPU
```

```

run.
5     """
6     from types import SimpleNamespace
7
8     import pytest
9
10    from backend.pipeline.detectors import fusion_features as ff
11
12
13    def _shuttle(frame, x, y, visible=True):
14        return SimpleNamespace(frame=frame, x=x, y=y, visible=visible)
15
16
17    # Square unit polygon and a left-half polygon (x in [0, 0.5]).
18    FULL = [(0.0, 0.0), (1.0, 0.0), (1.0, 1.0), (0.0, 1.0)]
19    LEFT_HALF = [(0.0, 0.0), (0.5, 0.0), (0.5, 1.0), (0.0, 1.0)]
20
21
22    # ----- point_in_polygon
23
24    def test_point_in_polygon_inside_outside():
25        assert ff.point_in_polygon((0.5, 0.5), FULL) is True
26        assert ff.point_in_polygon((1.5, 0.5), FULL) is False
27        assert ff.point_in_polygon((0.2, 0.5), LEFT_HALF) is True
28        assert ff.point_in_polygon((0.8, 0.5), LEFT_HALF) is False
29
30
31    def test_point_in_polygon_degenerate():
32        assert ff.point_in_polygon((0.5, 0.5), None) is False
33        assert ff.point_in_polygon((0.5, 0.5), [(0.0, 0.0), (1.0, 1.0)]) is False # <3
verts
34
35
36    # ----- person_in_court
37
38    def test_person_in_court_uses_foot_point_and_polygon():
39        # box foot = ((x1+x2)/2/W, y2/H). W=H=100.
40        in_box = (10, 10, 30, 30) # foot (0.2, 0.3) ? left half
41        out_box = (60, 10, 80, 30) # foot (0.7, 0.3) ? right half
42        assert ff.person_in_court(in_box, 100, 100, LEFT_HALF) is True
43        assert ff.person_in_court(out_box, 100, 100, LEFT_HALF) is False
44        # None polygon => everyone counts
45        assert ff.person_in_court(out_box, 100, 100, None) is True
46        # Unusable frame dims => foot (0,0); with a polygon that excludes origin => False
47        assert ff.person_in_court(in_box, 0, 0, LEFT_HALF) is True # (0,0) is on left half
edge-inside
48
49
50    # ----- counts / ratios
51
52    def test_in_court_counts_and_players_and_ratio():
53        near = (10, 10, 30, 30) # foot (0.2,0.3) in LEFT_HALF
54        far = (60, 10, 80, 30) # foot (0.7,0.3) NOT in LEFT_HALF
55        per_frame = {0: [(1, near), (2, far)], 1: [(1, near)], 2: [], 3: [(1, near), (2,
near)]}]
56
57        # With LEFT_HALF mask: counts = [1, 1, 0, 2]
58        counts = ff.in_court_counts(per_frame, 100, 100, LEFT_HALF)
59        assert counts == [1, 1, 0, 2]
60        assert ff.players_in_court(counts) == 1.0 # median of [0,1,1,2]
61        assert ff.in_court_ratio(counts, min_players=2) == pytest.approx(0.25)
62        assert ff.in_court_ratio(counts, min_players=1) == pytest.approx(0.75)
63        # No mask: far counts too -> [2,1,0,2]
64        assert ff.in_court_counts(per_frame, 100, 100, None) == [2, 1, 0, 2]
65        assert ff.players_in_court([]) == 0.0
66        assert ff.in_court_ratio([]) == 0.0

```

```

66
67
68 # ----- mean_player_motion
69
70 def test_mean_player_motion():
71     # track 1 moves 10px in x between frames; W=H=100 -> normalized disp 0.10.
72     per_frame = {0: [(1, (0, 0, 10, 10))], 1: [(1, (10, 0, 20, 10))]}
73     assert ff.mean_player_motion(per_frame, 100, 100) == pytest.approx(0.10)
74     # single frame -> 0; no shared track id -> 0
75     assert ff.mean_player_motion({0: [(1, (0, 0, 10, 10))]}, 100, 100) == 0.0
76     assert ff.mean_player_motion({0: [(1, (0, 0, 10, 10))], 1: [(2, (10, 0, 20, 10))]},
100, 100) == 0.0
77
78
79 # ----- two_sided_motion
80
81 def test_two_sided_motion():
82     net = ("y", 0.5)
83     near = (10, 10, 30, 40)      # foot y = 0.4 < 0.5 (near side)
84     far = (10, 60, 30, 90)      # foot y = 0.9 >= 0.5 (far side)
85     both = {0: [(1, near), (2, far)], 1: [(1, near)]}      # frame 0 two-sided, frame 1
one-sided
86     assert ff.two_sided_motion(both, 100, 100, net, None) == pytest.approx(0.5)
87     one_sided = {0: [(1, near)], 1: [(2, near)]}
88     assert ff.two_sided_motion(one_sided, 100, 100, net, None) == 0.0
89     assert ff.two_sided_motion(both, 100, 100, None, None) == 0.0    # no net -> 0
90
91
92 # ----- shuttle_player_colocation
93
94 def test_shuttle_player_colocation():
95     box = (40, 40, 60, 60)
96     per_frame = {0: [(1, box)], 1: [(1, box)], 2: [(1, box)]}
97     # frame0 shuttle inside box (held), frame1 shuttle far (in-flight), frame2
invisible.
98     shuttle = [_shuttle(0, 50, 50, True), _shuttle(1, 95, 95, True), _shuttle(2, 50, 50,
False)]
99     # aligned visible frames = {0, 1}; colocated = {0} -> 0.5
100     assert ff.shuttle_player_colocation(per_frame, shuttle, 100, 100) ==
pytest.approx(0.5)
101     # all-held -> 1.0
102     held = [_shuttle(0, 50, 50, True), _shuttle(1, 50, 50, True)]
103     assert ff.shuttle_player_colocation(per_frame, held, 100, 100) == pytest.approx(1.0)
104     # no shuttle -> 0.0; shuttle frame not person-sampled -> skipped (no aligned -> 0.0)
105     assert ff.shuttle_player_colocation(per_frame, [], 100, 100) == 0.0
106     assert ff.shuttle_player_colocation(per_frame, [_shuttle(99, 50, 50, True)], 100,
100) == 0.0
107
108
109 # ----- aggregator
110
111 def test_compute_person_features_keys():
112     near = (10, 10, 30, 40)
113     far = (10, 60, 30, 90)
114     per_frame = {0: [(1, near), (2, far)], 1: [(1, near), (2, far)]}
115     shuttle = [_shuttle(0, 20, 25, True)]
116     feats = ff.compute_person_features(per_frame, shuttle, 100, 100,
court_polygon=None, net=("y", 0.5))
117     assert set(feats) == set(ff.PERSON_FEATURE_NAMES)
118     assert all(isinstance(v, float) for v in feats.values())
119     assert feats["two_sided_motion"] == pytest.approx(1.0)    # both sides occupied every
frame
120
121
122
123 def test_compute_person_features_empty():

```

```

124     feats = ff.compute_person_features({}, [], 100, 100)
125     assert set(feats) == set(ff.PERSON_FEATURE_NAMES)
126     assert all(v == 0.0 for v in feats.values())
127

```

4. user (2026-06-13T19:15:48.789Z)

```

1     """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3     NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4     math is unit-testable in a GPU-free dev container. The detection (person boxes,
5     shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6     module only turns those signals into a handful of scale/translation-invariant
7     features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8     memorise this court/camera and fail to generalise off a small labelled set).
9
10    Conventions:
11    - A person box is ``(x1, y1, x2, y2)`` in pixels; ``(track_id, box)`` when motion
needs it.
12    - ``per_frame`` is ``{frame_idx: [(track_id, box), ...]}`` keyed by the ORIGINAL video
frame
13    index (so it aligns directly with the shuttle trajectory's ``.frame`` ? no clip re-
timing).
14    - A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
trajectory shape.
15    - ``court_polygon`` is normalised 0-1 ``[(x, y), ...]`` or ``None`` (None ? no mask,
every
16    detection counts ? the player-count-only mode).
17    - ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
0.0).
18
19    Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20    floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
21    """
22    from __future__ import annotations
23
24    from typing import Dict, List, Optional, Sequence, Tuple
25
26    BBox = Tuple[float, float, float, float]
27    Net = Tuple[str, float]
28
29
30    def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31        """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32        (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33        if polygon is None or len(polygon) < 3:
34            return False
35        x, y = point
36        inside = False
37        n = len(polygon)
38        j = n - 1
39        for i in range(n):
40            xi, yi = polygon[i]
41            xj, yj = polygon[j]
42            # Does the horizontal ray at y cross edge (i, j)?
43            if (yi > y) != (yj > y):
44                x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45                if x < x_cross:
46                    inside = not inside
47            j = i
48        return inside
49
50

```

```

51     def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52         """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53         the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54         if frame_width <= 0 or frame_height <= 0:
55             return (0.0, 0.0)
56         x1, _y1, x2, y2 = box
57         return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60     def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61         if frame_width <= 0 or frame_height <= 0:
62             return (0.0, 0.0)
63         x1, y1, x2, y2 = box
64         return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67     def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69         """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70         True (count everyone) when it is None."""
71         if court_polygon is None:
72             return True
73         return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
74
75
76     def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
77                        frame_height: float,
78                        court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
79         """Per-frame count of in-court persons (one entry per sampled frame)."""
80         return [
81             sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
court_polygon))
82             for _frame_idx, dets in sorted(per_frame.items())
83         ]
84
85
86     def _median(vals: Sequence[float]) -> float:
87         if not vals:
88             return 0.0
89         s = sorted(vals)
90         n = len(s)
91         return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
92
93
94     def players_in_court(counts: Sequence[int]) -> float:
95         """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
96         return _median(list(counts))
97
98
99     def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
100         """Fraction of sampled frames with >= `min_players` in court (track stability vs
flicker)."""
101         if not counts:
102             return 0.0
103         return sum(1 for c in counts if c >= min_players) / len(counts)
104
105
106     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
107                           frame_width: float, frame_height: float) -> float:
108         """Mean per-track centre displacement between consecutive sampled frames, normalised
109         by the frame width (resolution-invariant). 0.0 if <2 frames or no shared tracks."""
110         if frame_width <= 0 or len(per_frame) < 2:

```

```

111         return 0.0
112     frames = sorted(per_frame.keys())
113     steps: List[float] = []
114     for a, b in zip(frames, frames[1:]):
115         prev = {tid: box for tid, box in per_frame[a]}
116         for tid, box in per_frame[b]:
117             if tid in prev:
118                 cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
119                 cx1, cy1 = _center_norm(box, frame_width, frame_height)
120                 steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
121     if not steps:
122         return 0.0
123     return sum(steps) / len(steps)
124
125
126 def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
127                     frame_height: float, net: Optional[Net],
128                     court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
129     """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
130
131     A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
132     The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
133     0.0.
134
135     """
136     if net is None or not per_frame:
137         return 0.0
138     axis, pos = net
139     both = 0
140     for _frame_idx, dets in per_frame.items():
141         near, far = False, False
142         for _tid, box in dets:
143             if not person_in_court(box, frame_width, frame_height, court_polygon):
144                 continue
145             fx, fy = _foot_norm(box, frame_width, frame_height)
146             coord = fx if axis == "x" else fy
147             if coord < pos:
148                 near = True
149             else:
150                 far = True
151         if near and far:
152             both += 1
153     return both / len(per_frame)
154
155 def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
156                    pad_frac: float) -> bool:
157     """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
158     width/height (the 'adjacent / in-hand' tolerance)?"""
159     x1, y1, x2, y2 = box
160     px = pad_frac * frame_width
161     py = pad_frac * frame_height
162     return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
163
164 def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
165                             frame_width: float, frame_height: float,
166                             pad_frac: float = 0.03) -> float:
167     """Fraction of (visible shuttle frames that were also person-sampled) in which the
168     shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
169     ~0**
170     ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
171     """
172     if not per_frame or frame_width <= 0 or frame_height <= 0:

```

```

172         return 0.0
173     aligned = 0
174     colocated = 0
175     for p in shuttle_points or []:
176         if not getattr(p, "visible", False):
177             continue
178         dets = per_frame.get(int(p.frame))
179         if dets is None:          # shuttle frame wasn't person-sampled ? can't judge
180             continue
181         aligned += 1
182         if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
183             for _tid, box in dets):
184             colocated += 1
185     return colocated / aligned if aligned else 0.0
186
187
188     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
189     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
190     PERSON_FEATURE_NAMES = [
191         "players_in_court", "two_sided_motion", "mean_player_motion",
192         "in_court_ratio", "shuttle_player_colocation",
193     ]
194
195
196     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
197 shuttle_points,
198                                frame_width: float, frame_height: float,
199                                court_polygon: Optional[Sequence[Tuple[float, float]]] =
200 None,
201                                net: Optional[Net] = None,
202                                min_players: int = 2) -> Dict[str, float]:
203         """Aggregate the per-window person features from per-frame detections + the in-
204 window
205 shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
206         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
207         return {
208             "players_in_court": players_in_court(counts),
209             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
210 court_polygon),
211             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
212             "in_court_ratio": in_court_ratio(counts, min_players),
213             "shuttle_player_colocation": shuttle_player_colocation(
214                 per_frame, shuttle_points, frame_width, frame_height),
215         }
216
217

```

5. assistant (2026-06-13T19:15:54.195Z)

Now let me verify the key claims in the finding. Let me trace through the ray-cast at the origin (0,0) for LEFT_HALF, and check finding #2's claim (fh=0 collapse).

The finding makes specific claims I need to verify:

1. With frame dims 0, `\_foot\_norm` returns (0.0, 0.0) ? confirmed at lines 54-55.
2. `point\_in\_polygon((0,0), LEFT\_HALF)` returns True via ray-cast.
3. The test asserts this as if intended behavior.

Let me trace the ray-cast manually for point (0,0) against LEFT_HALF = [(0,0),(0.5,0),(0.5,1),(0,1)].

6. user (2026-06-13T19:15:58.282Z)

```

foot_norm dims0: (0.0, 0.0)
point_in_polygon (0,0) LEFT_HALF: True
person_in_court dims0: True
person_in_court out_box dims0: True

```

7. assistant (2026-06-13T19:16:05.458Z)

The factual claims in the finding are correct:

- `\_foot\_norm` returns (0,0) for degenerate dims
- `point\_in\_polygon((0,0), LEFT\_HALF)` returns True
- `person\_in\_court(in\_box, 0, 0, LEFT\_HALF)` is True (as asserted on line 47)

Crucially, my last probe shows the comment "(0,0) is on left half edge-inside" describes an artifact, not a designed contract: with degenerate dims, EVERY box collapses to (0,0) and is counted in-court, regardless of where the person actually is. The `out\_box` (which is False with valid dims on line 43) also becomes True with dims 0. So the test pins "all detections counted in-court when dims are unusable."

Now let me check the diff for the edited files to understand whether degenerate dims is a real reachable path and whether the finding's severity assessment is fair. Let me also confirm finding #2's "fh=0 collapse" claim referenced here.

8. user (2026-06-13T19:16:07.368Z)

---untracked check---

```
?? backend/pipeline/detectors/fusion_features.py
?? backend/pipeline/segmenters/fusion_hybrid.py
```

9. assistant (2026-06-13T19:16:11.440Z)

Both are new untracked files. Let me check how `compute\_person\_features` is called in the segmenter to see whether degenerate frame dims (0) is even a reachable production path, which bears on whether this is a meaningful behavior to pin.

10. user (2026-06-13T19:16:12.621Z)

```
71:                                     frame_width: float, video_path: str,
73:         stats = super()._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
idx_cfg)
82:         stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
86:                                     frame_width: float, video_path: str,
99:         fw = det["frame_width"] or frame_width
100:         fh = det["frame_height"] or 0.0
106:         return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
```

11. assistant (2026-06-13T19:16:16.524Z)

This is the key context. Let me read that section of `fusion\_hybrid.py` to see how `fw`/`fh` get set and whether 0 is reachable.

12. user (2026-06-13T19:16:17.379Z)

```
84
85         def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
86                               frame_width: float, video_path: str,
87                               fcfg: Dict[str, Any]) -> Dict[str, float]:
88             """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
89             frame range and compute the scale/translation-invariant person features. Lazily
builds
90             ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
91             # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
92             from backend.pipeline.detectors.person_detector import PersonDetector
93             if self._person is None:
94                 self._person = PersonDetector(self.config)
95
```

```

96         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
97         start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)
98         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
99         fw = det["frame_width"] or frame_width
100        fh = det["frame_height"] or 0.0
101
102        poly = fcfg.get("court_polygon")
103        polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
104        net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
105        shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
106        return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
107                                         court_polygon=polygon, net=net)
108
109    def _select_for_handoff(self, windows: List[Dict[str, Any]],
110                           window_stats: List[Dict[str, Any]],
111                           idx_cfg: Dict[str, Any]) -> List[int]:
112        """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
113        (0) keep every window ? identical to the substrate. Persisted candidates are

```

13. assistant (2026-06-13T19:16:26.650Z)

This is highly relevant. Line 99-100 reveals the real production behavior:

- `fw = det["frame\_width"] or frame\_width` ? falls back to the passed `frame\_width` if detector returns 0/None.
- `fh = det["frame\_height"] or 0.0` ? falls back to `**0.0**`, NOT to a real frame height.

So `fh` can genuinely be 0 in production if the person detector returns a falsy `frame\_height` (e.g., the detector failed to read dims, or returned 0). This confirms finding #2's "fh=0 collapse" is a real reachable path ? and when `fh=0`, `\_foot\_norm` returns (0,0) for every box, and with a court polygon every detection collapses to the origin. The behavior the test pins (line 47) is exactly the production degenerate path.

Now let me assess the finding's actual claim carefully. The finding is specifically about the TEST assertion on line 47 ? that it "locks in a dubious artifact as if it were intended." Let me re-read line 46-47 and its comment precisely.

The test comment on line 46 says: "Unusable frame dims => foot (0,0); with a polygon that excludes origin => False" ? but then line 47 asserts `is True`, and the inline comment says "(0,0) is on left half edge-inside."

This is a notable internal contradiction in the test itself: the line-46 comment sets up an expectation of "polygon that excludes origin => False," but LEFT_HALF does NOT exclude the origin (origin is a vertex of LEFT_HALF), so line 47 asserts True. The test author's own framing comment is misleading. Let me verify the origin is a vertex of LEFT_HALF and whether a polygon that truly excludes the origin would yield False.

14. user (2026-06-13T19:16:30.716Z)

```

origin in EXCL_ORIGIN: False
origin in RIGHT_HALF: False
person_in_court dims0 RIGHT_HALF: False

```

15. assistant (2026-06-13T19:16:38.351Z)

This confirms the core of the finding decisively. With degenerate dims (0,0), the in-court classification is entirely an accident of where the polygon happens to sit relative to the origin:

- LEFT_HALF (origin is a vertex/edge) ? True
- RIGHT_HALF or a polygon excluding origin ? False

So the "in-court" answer for a detection with no valid coordinates is determined by arbitrary polygon placement, not by any meaningful contract. The test asserts `True` for one such accidental case and frames it as "edge-inside" ? pinning an implementation artifact.

Let me now weigh whether this is a "nitpick/false-positive" (which would make it real=false). The finding's stated severity is "low," which is appropriate and honest ? it doesn't overclaim. Let me verify the test would still pass and that the assertion isn't vacuous (the finding correctly says it's "not vacuous").

16. user (2026-06-13T19:16:40.634Z)

. [100%]
1 passed in 0.06s

17. assistant (2026-06-13T19:16:44.363Z)

Let me do a final adversarial check: is there any chance `fh=0` is actually unreachable in production (which would weaken the "arguably wrong behavior" angle and make this purely a test-quality nitpick)? Let me check what `detections\_per\_frame` returns for `frame\_height`.

18. user (2026-06-13T19:16:46.019Z)

```
8:         return windows
10:+     def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
16:+         per-frame boxes), this returns the raw per-frame detections so the fusion features
can
18:+         (seek + sequential read), NOT a re-extracted clip, so the returned frame indices
match
22:+         Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
23:+         "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-seek
31:+         return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
36:+         frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
37:+         frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
54:+         return {"frames": frames, "fps": fps,
55:+             "frame_width": float(frame_width), "frame_height": float(frame_height)}
```

19. assistant (2026-06-13T19:16:55.711Z)

This seals it. Line 31 of the diff shows `detections\_per\_frame` returns `frame\_height: 0.0` when the video can't be opened (the early-return path). In that case, in `fusion\_hybrid.py`:
- `fh = det["frame\_height"] or 0.0` ? 0.0
- And `det["frames"]` is `{}` so `compute\_person\_features` returns all-zeros anyway in THAT specific path.

But more importantly, `frame\_height` comes from `cv2.CAP\_PROP\_FRAME\_HEIGHT` (line 37), which can legitimately return 0.0 on some codecs/containers even when frames ARE read successfully (a well-known OpenCV quirk). In that case `det["frames"]` is non-empty but `fh=0`, so every box collapses to foot (0,0) and ? with a court polygon positioned like LEFT_HALF ? gets silently counted as in-court. So the degenerate behavior pinned by line 47 is a genuinely reachable production path, not a synthetic-only curiosity.

Verdict: The finding holds. It is accurate on every factual claim, correctly characterizes the assertion as pinning an incidental ray-cast artifact rather than an intended contract, and its proposed fix (decide and document the degenerate-dims contract explicitly) is sound. The "low" severity is honest ? the assertion is real (not vacuous), the test passes, and it's a test-quality/design-clarity issue rather than a functional bug. This is not a nitpick because it documents a genuinely under-specified and arguably-wrong degenerate behavior that is reachable in production via cv2 returning a 0 frame height.

20. user (2026-06-13T19:17:10.892Z)

Structured output provided successfully

21. assistant (2026-06-13T19:17:19.409Z)

I verified this finding against the actual code and confirmed it is real (severity: low).

Key evidence at `C:/Users/avidu/Projects/badminton-highlight-indexer`:

- ``tests/test_fusion_features.py:47`` asserts ``person_in_court(in_box, 0, 0, LEFT_HALF)` is `True``. I confirmed at runtime that ``_foot_norm`` returns `(0,0)` for `dims 0` (``fusion_features.py:54-55``) and the ray-cast classifies `(0,0)` as inside `LEFT_HALF`.
- The assertion pins an artifact, not a contract: with `dims 0` every box collapses to `(0,0)`, so the answer is decided purely by polygon placement. I proved ``out_box`` (`False` with valid `dims` at line 43) becomes `True` at `dims 0`, and a `RIGHT_HALF/origin-excluding` polygon yields `False`. So "in-court" for a coordinate-less detection is accidental.
- The path is reachable in production: ``fusion_hybrid.py:100`` does ``fh = det["frame_height"]` or `0.0``, and ``frame_height`` comes from ``cv2.CAP_PROP_FRAME_HEIGHT`` (``person_detector.py`` diff line 37), which can return `0.0` on some codecs even with readable frames ? collapsing all boxes to the origin and silently counting them on-court.

The "low" rating is honest: the assertion is non-vacuous, the test passes, and this is a test-quality/under-specified-contract issue rather than a functional bug. Not a nitpick, because it locks in a genuinely dubious, reachable degenerate behavior. The proposed fix (decide and document the degenerate-dims contract explicitly ? ideally treat bad dims as not-in-court) is sound.